Telco Customer Churn Prediction

End-to-End Machine Learning Project — Final Report

Dataset: Telco Customer Churn (7,043 customers, 21 columns) | **Target:** Churn (Yes/No) | **Task:** binary classification

Companion files: telco_churn_project.ipynb (full code, outputs and observations), **app.py** (Streamlit app), **churn_model.joblib** (saved model).

Summary

This project builds a complete machine learning pipeline that predicts whether a telecom customer will churn. After cleaning the data (including 11 hidden missing values in **TotalCharges**), exploring it in depth and engineering features, **12 models** were trained and compared: six individual classifiers and six ensemble methods (Bagging, three Boosting variants, Voting and Stacking). The best model by F1-score was the **Random Forest** (F1 = 0.640, recall = 0.778, precision = 0.543, accuracy = 0.767, ROC-AUC = 0.844 on a held-out test set of 1,409 customers). It was deployed in a Streamlit application.

The strongest churn signals are a **month-to-month contract**, **short tenure**, **fiber-optic internet**, **electronic-check payment** and the **absence of security / tech-support add-ons**.

Part	Content
1	Data understanding
2	Data cleaning & preprocessing
3	Exploratory data analysis & insights
4–5	Feature engineering and train-test split
6–7	Model building and ensemble learning
8	Model evaluation
9	Conclusion
10	Streamlit application

Part 1 — Data Understanding

The dataset contains **7,043 rows and 21 columns**: 4 demographic columns, 4 account columns, 9 service columns, 2 billing columns, the identifier **customerID** and the target **Churn**.

Aspect	Result
Dimensions	7,043 rows × 21 columns
Numerical features	**tenure**, **MonthlyCharges**, **TotalCharges** (stored as text!), plus **SeniorCitizen** stored as a 0/1 flag
Categorical features	16 columns (gender, Partner, Dependents, PhoneService, MultipleLines, InternetService, six add-on services, Contract, PaperlessBilling, PaymentMethod, …)
Missing values	0 true NaN, but **11 blank strings** in **TotalCharges** — all customers with **tenure = 0** (new, not yet billed)
Duplicates	0 duplicate rows, 0 duplicate customer IDs
Target	**Churn**: 5,174 No (73.5%) and 1,869 Yes (26.5%) — imbalanced binary target

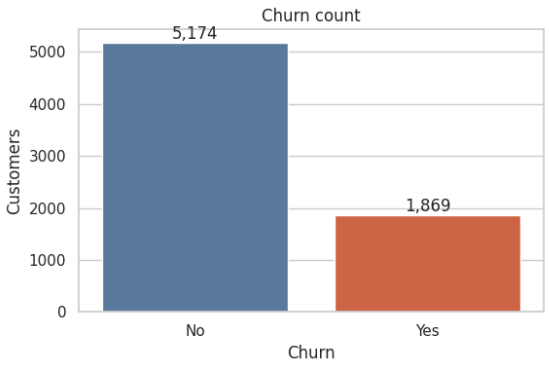
Target variable. Churn indicates whether the customer left the company. Predicting it lets the business target retention offers at customers who are likely to leave. Because only about 1 in 4 customers churns, accuracy alone is misleading and recall / F1 / ROC-AUC are used for model selection.

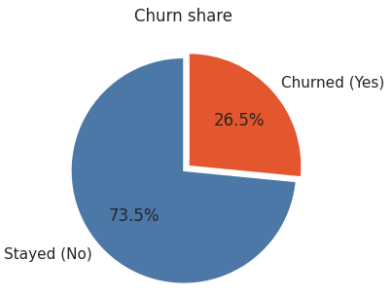
Part 2 — Data Cleaning & Preprocessing

Step	What was done	Why
Rename columns	**customerID** → **customer_id**, **SeniorCitizen** → **senior_citizen**, … (snake_case)	Consistent, typo-safe names
Strip whitespace	Removed leading/trailing spaces in all text columns	Hidden spaces create fake categories and the blank **TotalCharges** cells
Convert type + missing values	**total_charges** converted to float; the 11 blanks filled with **0**	All 11 are customers with tenure 0 who have not been billed, so 0 is the true value; mean/median would invent charges and dropping rows loses valid customers
Duplicates	Checked and removed (0 found)	Duplicates can leak between train and test sets
Drop identifier	Removed **customer_id**	Unique per row, no predictive value
Inconsistent categories	**No phone service** and **No internet service** merged into **No**; **senior_citizen** 0/1 → No/Yes	Those labels duplicate **phone_service** / **internet_service**; one clean Yes/No convention
Outliers	IQR rule on 3 numeric features: **0 outliers**	All values are valid business values, nothing removed
Encoding & scaling	Done in Part 4, after the split where needed	Scaling is fitted on the training set only to avoid data leakage

Part 3 — Exploratory Data Analysis & Insights

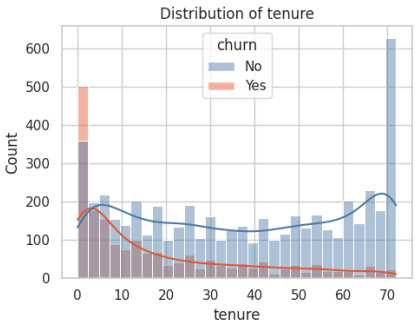
3.1 Target distribution

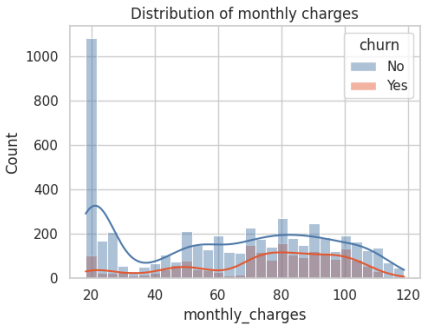


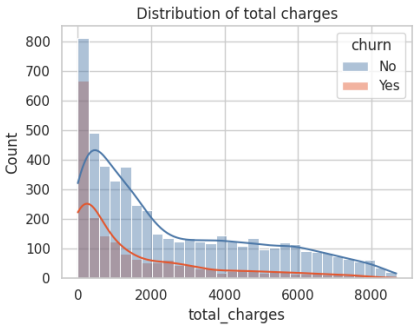


Observation: The chart shows how many customers stayed and churned. 73.5% stayed and 26.5% (1,869) churned — about 2.8 stayers per churner. A model that always predicts "No" would reach 73.5% accuracy while finding no churners, so recall, F1 and ROC-AUC are the key metrics and stratified splitting / class weighting are needed.

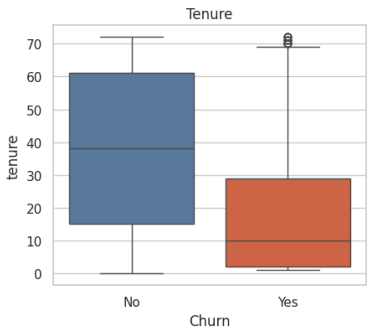
3.2 Numerical features

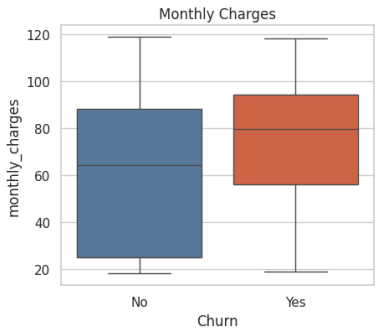


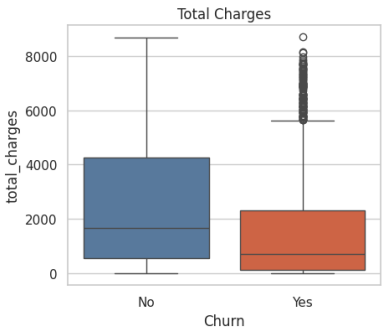




Observation: Tenure is U-shaped (many very new and many very loyal customers); churners concentrate at low tenure. Monthly charges are bimodal (a large group paying about $20, the rest $50–110) and churners lean to the higher prices. Total charges are right-skewed. Short tenure and high price signal risk; the skew motivates scaling.

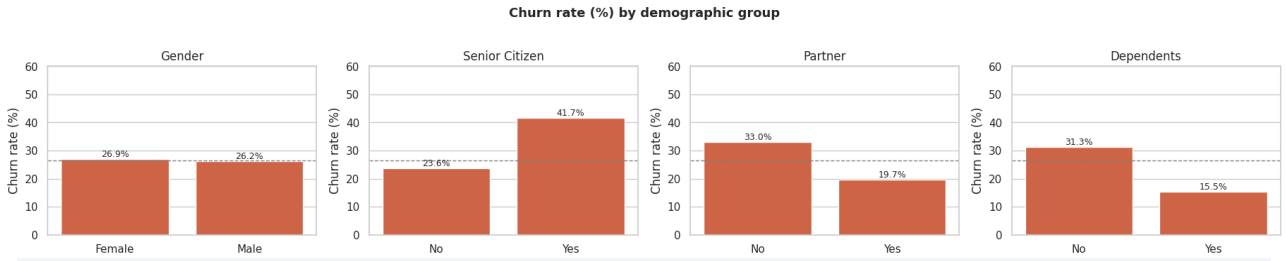






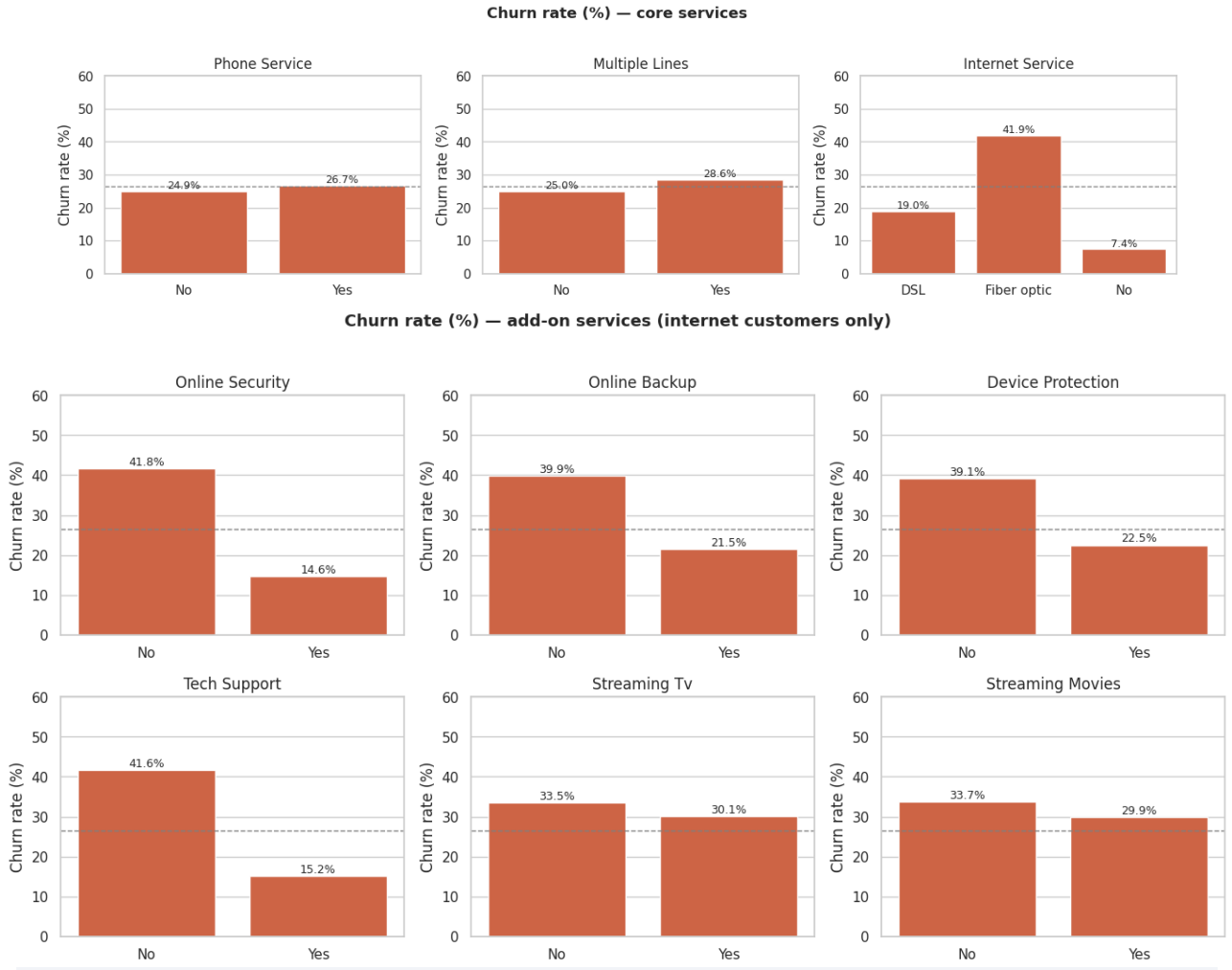
Observation: The median tenure of churners is 10 months vs 38 for stayers; median monthly charges are $79.65 vs $64.43; median total charges $704 vs $1,680. No IQR outliers exist. Tenure separates the groups most clearly.

3.3 Demographics



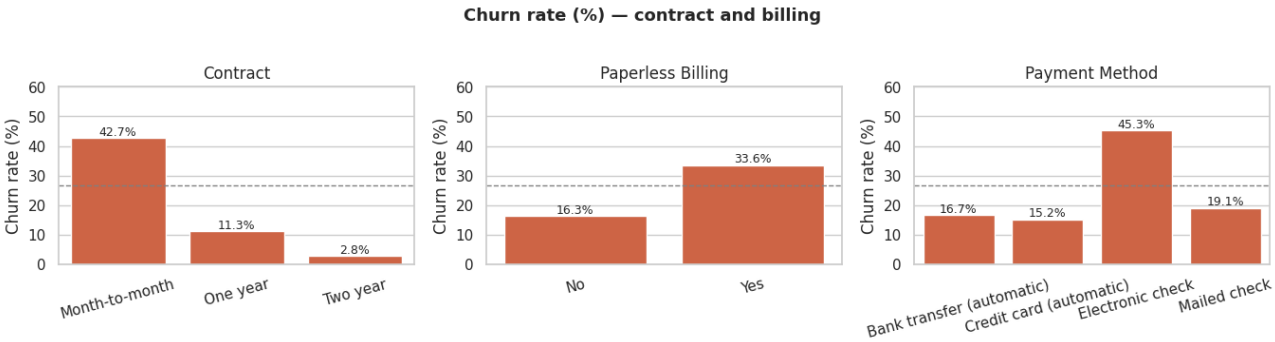
Observation: Gender is irrelevant (26.9% vs 26.2%). Senior citizens churn far more (41.7% vs 23.6%), and customers without a partner (33.0% vs 19.7%) or dependents (31.3% vs 15.5%) churn more.

3.4 Services



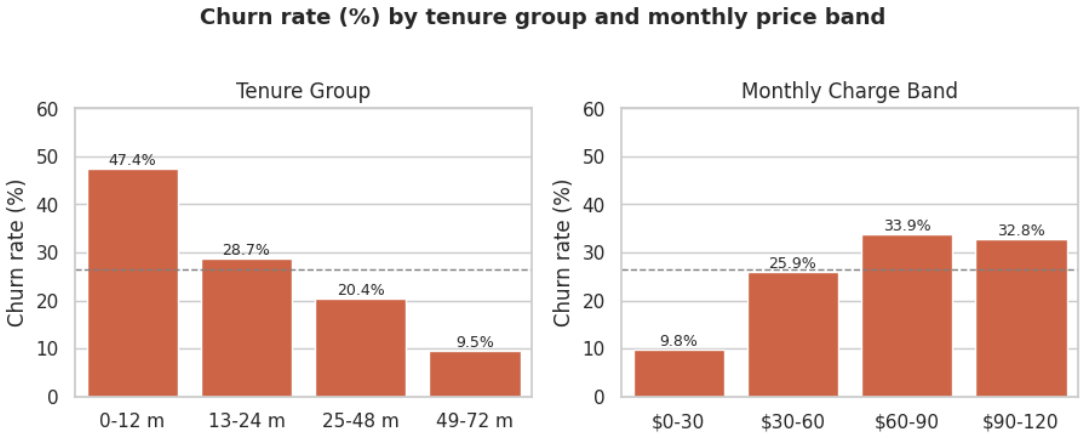
Observation: Fiber-optic customers churn at 41.9% (DSL 19.0%, no internet 7.4%). Among internet customers, online security (14.6% vs 41.8%), tech support (15.2% vs 41.6%), online backup (21.5% vs 39.9%) and device protection (22.5% vs 39.1%) strongly reduce churn, while streaming services and phone/multiple lines barely matter. These are the services that reduce churn.

3.5 Contract and billing



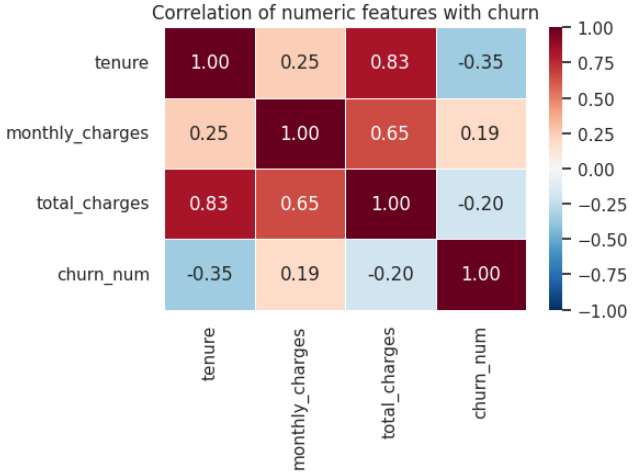
Observation: Month-to-month customers churn at 42.7% vs 11.3% (one year) and 2.8% (two years) and make up 88.6% of all churners. Electronic-check payers churn at 45.3% vs 15–19% for the other methods; paperless billing 33.6% vs 16.3%. Commitment and payment friction are strong early-warning signs.

3.6 Tenure and price bands



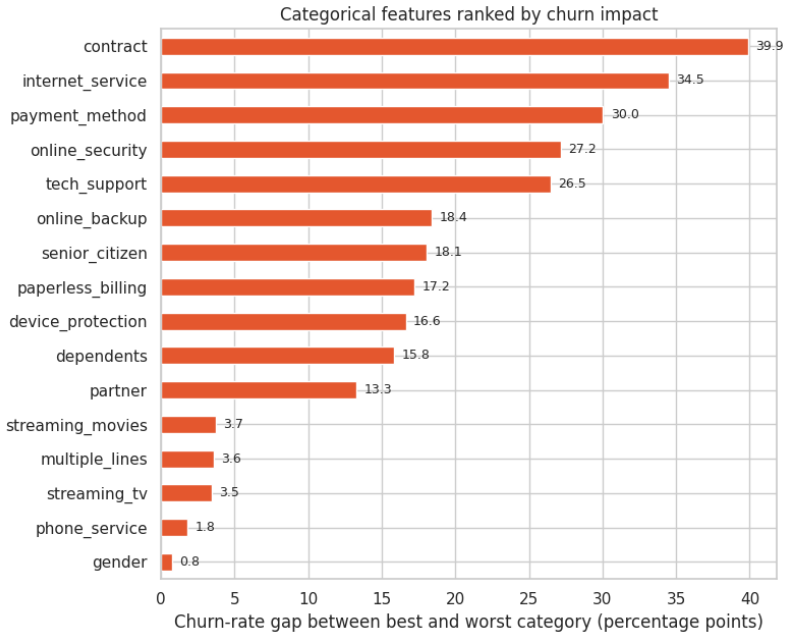
Observation: Churn falls steadily with tenure: 47.4% in the first year, 28.7%, 20.4%, then 9.5% after four years. 55.5% of all churners leave within 12 months. Customers paying under $30 churn at only 9.8% vs 33–34% for the $60–120 bands (largely a fiber-optic effect).

3.7 Correlation heatmap



Observation: Tenure has the strongest correlation with churn (r = −0.35), then total charges (−0.20) and monthly charges (+0.19). Tenure and total charges correlate at 0.83, so they overlap in information (multicollinearity), which matters for linear-model interpretation but not for tree models.

3.8 Which features matter most?



Observation: Contract (≈ 40 points of churn-rate gap), internet service (≈ 35), payment method (≈ 30), online security and tech support (≈ 27 and 26) are the strongest categorical drivers; gender, phone service, multiple lines and streaming are at the bottom (1–4 points).

3.9 Unusual observations

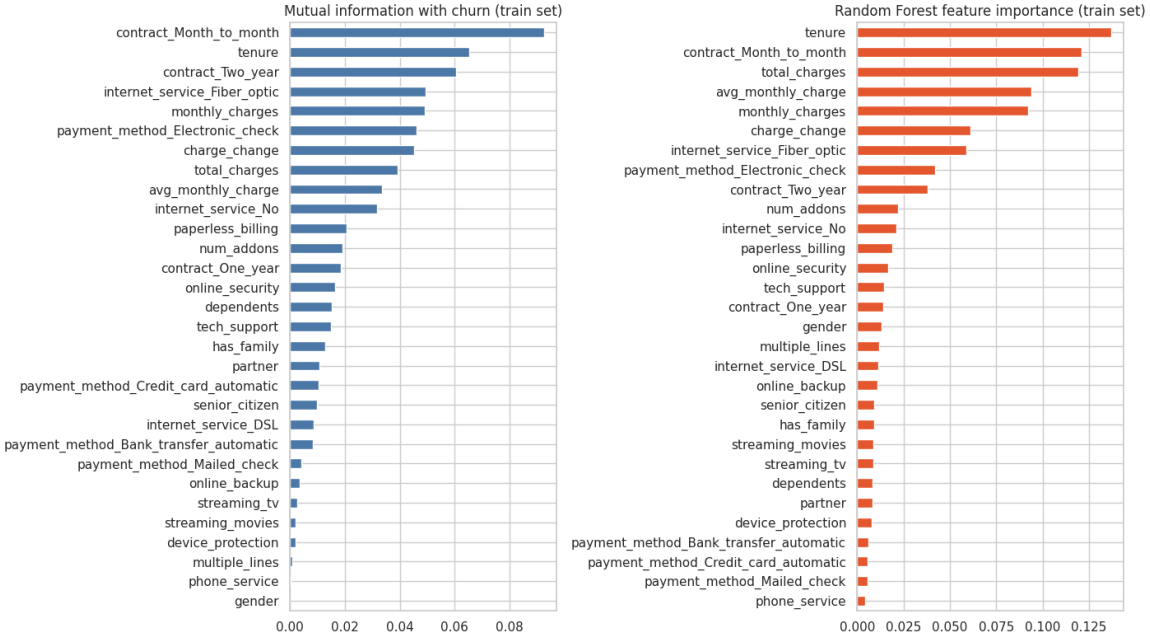
Observation: (1) 11 hidden blanks belong to brand-new customers; none churned. (2) Price is driven by internet type (median about $20 without internet, $56 DSL, $92 fiber), so "high price → churn" is largely a fiber effect. (3) Month-to-month + fiber + electronic-check customers (1,307) churn at 60.4%, whereas customers on 1–2-year contracts (3,168) churn at only 6.8%.

3.10 Final EDA summary

1. **Imbalance:** only 26.5% of customers churn, so recall/F1/AUC matter more than accuracy.
2. **Contract is the strongest driver:** 42.7% churn on month-to-month vs 2.8% on two-year contracts; 88.6% of churners are month-to-month.
3. **Tenure protects:** 47.4% churn in year one vs 9.5% after four years; 55.5% of churners leave within 12 months.
4. **Fiber optic and electronic check are high risk** (41.9% and 45.3%); together with a month-to-month contract the rate reaches 60.4%.
5. **Security and support add-ons cut churn** from about 42% to about 15% among internet customers; streaming add-ons do not.
6. **Demographics:** seniors (41.7%), customers without partner or dependents churn more; gender has no effect.
7. **Price:** churners pay more per month, but this mostly reflects fiber-optic service; the cheapest phone-only customers churn at only 9.8%.
8. **Data quality:** no duplicates, no outliers; only 11 hidden blanks in **total_charges**.

Part 4 — Feature Engineering

Technique	Application	Why
Feature creation	**num_addons**, **avg_monthly_charge**, **charge_change**, **has_family**	Express engagement, historic price and price change; **charge_change** ranks 6th–7th in importance although its linear correlation with churn is ≈ 0
Label encoding	Binary columns (Yes/No, Female/Male) and the target (No = 0, Yes = 1)	Two categories imply no false ordering
One-hot encoding	**internet_service**, **contract**, **payment_method** (fixed category lists)	Nominal columns; avoids inventing an order; identical encoding at prediction time
Feature scaling	**StandardScaler** on 6 numeric/engineered columns, **fitted on the training set only**	KNN, SVM and Logistic Regression are sensitive to scale; avoids leakage
Feature selection	Mutual information + Random Forest importance on the training set; dropped **gender**, **phone_service**, **multiple_lines** (MI < 0.001)	Removes features with no signal; 27 of 30 features kept



Observation: Both rankings agree that month-to-month contract, tenure, the charge features, the engineered **charge_change**, fiber optic and electronic check are the most informative features. **gender**, **phone_service** and **multiple_lines** carry almost no information and were dropped.

Part 5 — Train-Test Split

80% training (5,634 customers) / 20% test (1,409 customers), **random_state = 42** for reproducibility and **stratify = y** so both sets keep a 26.5% churn rate (26.535% train, 26.544% test).

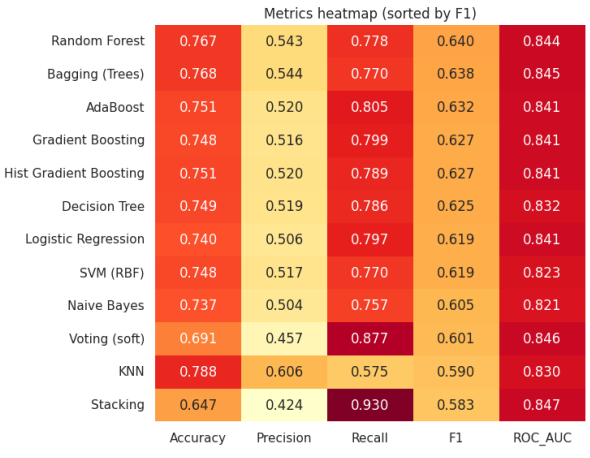
Part 6 & 7 — Model Building and Ensemble Learning

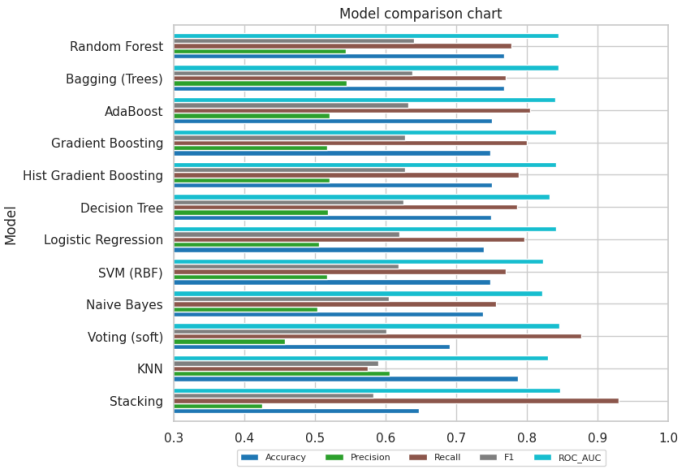
Six individual classifiers (Logistic Regression, Decision Tree, Random Forest, KNN, SVM-RBF, Gaussian Naive Bayes) and six ensembles — **Bagging** (100 trees), **Boosting** (AdaBoost, Gradient Boosting, Hist Gradient Boosting), a soft **Voting** classifier (Logistic Regression + Random Forest + Gradient Boosting) and the **Stacking** classifier (5 base models + Logistic Regression meta-learner, 5-fold out-of-fold predictions) — were trained on the same data. Balanced class weights were used wherever the algorithm supports them, to compensate for the 26.5% churn rate. All models use the default 0.5 threshold. Voting and Stacking were additionally fitted with balanced sample weights although their member models already use **class_weight="balanced"**, which most likely corrects for the imbalance twice (see Part 8).

Part 8 — Model Evaluation

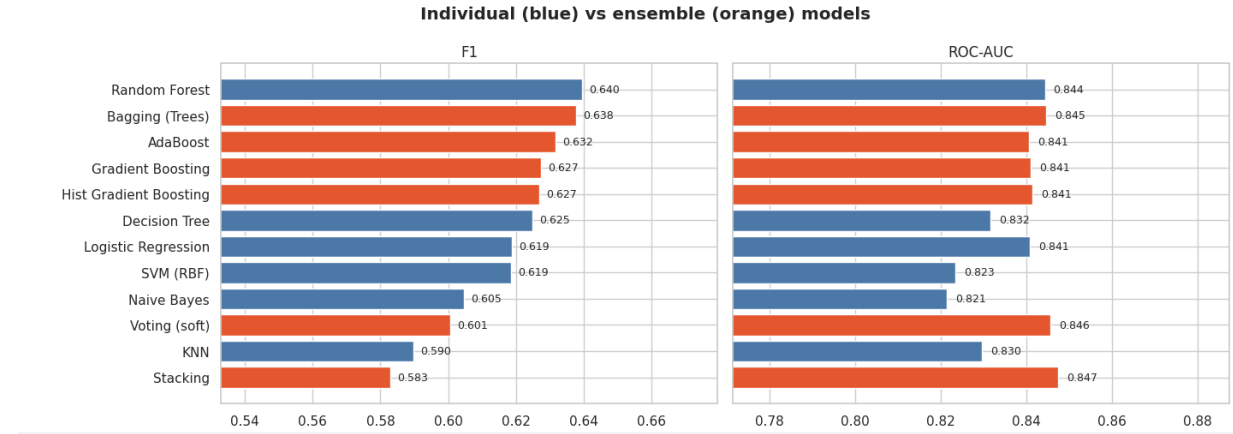
#	Model	Type	Accuracy	Precision	Recall	F1	ROC-AUC
1	Random Forest	Individual	0.767	0.543	0.778	0.640	0.844
2	Bagging (Trees)	Ensemble	0.768	0.544	0.770	0.638	0.845
3	AdaBoost	Ensemble	0.751	0.520	0.805	0.632	0.841
4	Gradient Boosting	Ensemble	0.748	0.516	0.799	0.627	0.841
5	Hist Gradient Boosting	Ensemble	0.751	0.520	0.789	0.627	0.841
6	Decision Tree	Individual	0.750	0.518	0.786	0.625	0.832
7	Logistic Regression	Individual	0.740	0.506	0.797	0.619	0.841
8	SVM (RBF)	Individual	0.748	0.517	0.770	0.619	0.823
9	Naive Bayes	Individual	0.737	0.504	0.757	0.605	0.821
10	Voting (soft)	Ensemble	0.691	0.457	0.877	0.601	0.846
11	KNN	Individual	0.788	0.606	0.575	0.590	0.830
12	Stacking	Ensemble	0.647	0.424	0.930	0.583	0.847

Test-set results (1,409 customers; churn = positive class), sorted by F1. Highlighted row = selected model.

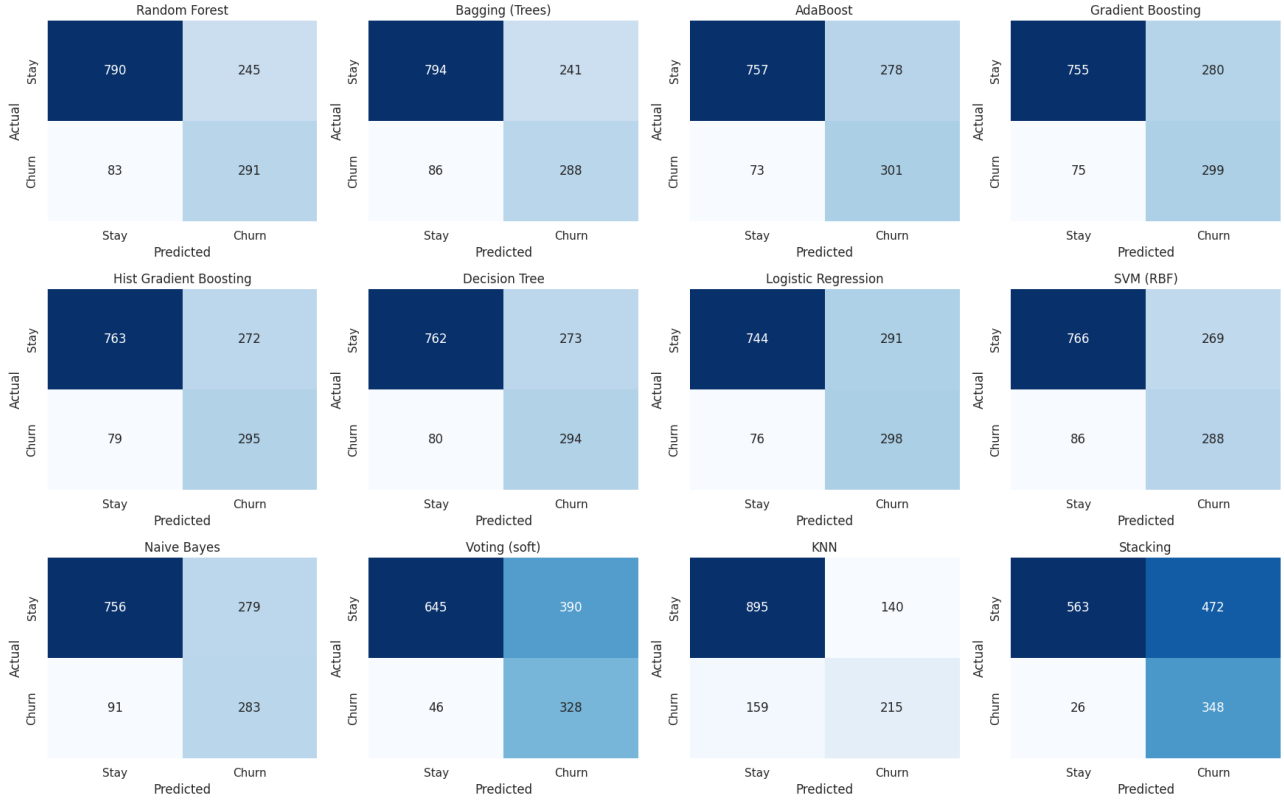




Observation: Random Forest has the best F1 (0.640), closely followed by Bagging (0.638), AdaBoost (0.632) and the other boosting models (0.627); Voting (0.601), KNN (0.590) and Stacking (0.583) are last. Stacking has the best ROC-AUC (0.847) and Voting is close behind (0.846), and they also have the highest recall (0.930 and 0.877), but at the 0.5 threshold they flag far too many customers (precision 0.424 and 0.457, accuracy 0.647 and 0.691), so their F1 is the lowest. A likely cause is that balanced sample weights were added on top of the members' own balanced class weights. KNN has the best accuracy (0.788) and precision (0.606) but only 0.575 recall, so it misses about 4 in 10 churners. Train and test accuracy are close for all models (no serious overfitting). Differences of about 0.01 are within noise for a test set with 374 churners.

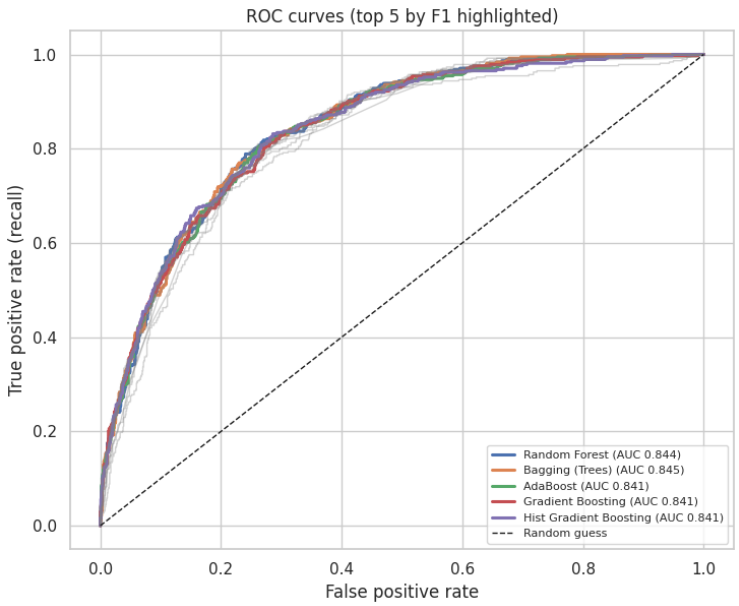


Observation: Mean F1 is almost identical for individual models (0.616) and ensembles (0.618), while mean ROC-AUC is higher for ensembles (0.843 vs 0.832). Bagging and the boosting models match the best individual model but do not clearly beat it (Random Forest is itself a bagging ensemble), and Voting and Stacking have the best AUC but a poor F1 at the 0.5 threshold.



Confusion matrices of all 12 models (rows = actual, columns = predicted).

Observation: Random Forest (best model): 790 true negatives, 245 false positives, 83 false negatives and 291 true positives — it catches 291 of 374 churners (recall 0.778) with precision 0.543. Class-weighted models accept more false alarms to catch more churners; KNN has the fewest false positives but the most missed churners, while Voting and Stacking flag the most customers as churners and therefore produce the most false alarms.



Observation: All models are far above random guessing (AUC 0.82–0.85) and the top curves nearly overlap, i.e. they rank customers by churn risk almost identically. An AUC of about 0.84 means a random churner gets a higher risk score than a random loyal customer about 84% of the time.

Part 9 — Conclusion

Most important preprocessing steps

- Found and fixed the **hidden missing values**: **total_charges** was stored as text and 11 blanks (all brand-new customers with tenure 0) were converted and filled with 0, which is exact rather than an estimate.
- Merged the redundant **No internet service** / **No phone service** labels into **No**, dropped the ID column, and checked duplicates (none) and outliers (none).
- Label-encoded binary columns and the target, one-hot encoded **internet_service**, **contract** and **payment_method**, and created four features (**num_addons**, **avg_monthly_charge**, **charge_change**, **has_family**); **charge_change** turned out to be one of the most informative features despite near-zero linear correlation.
- Avoided leakage: stratified 80/20 split with **random_state=42**, scaler and feature selection fitted on the training set only (3 useless features dropped, 27 kept), and balanced class weights to handle the 26.5% churn rate.

Key EDA insights

1. Only 26.5% of customers churn, so recall, F1 and ROC-AUC matter more than accuracy.
2. Contract type is the strongest driver: month-to-month customers churn at 42.7% vs 2.8% for two-year contracts and are 88.6% of all churners.
3. Churn falls with tenure (47.4% in year one, 9.5% after four years); 55.5% of churners leave within 12 months.
4. Fiber-optic (41.9%) and electronic-check (45.3%) customers are high risk; month-to-month + fiber + electronic check reaches 60.4%.
5. Online security and tech support cut churn from about 42% to about 15% among internet customers; gender has no effect.

Best-performing model: Random Forest

On the 1,409-customer test set it reached **F1 = 0.640, recall = 0.778, precision = 0.543, accuracy = 0.767 and ROC-AUC = 0.844**. It correctly flags 291 of the 374 churners and raises 245 false alarms.

Why this model was selected

- It has the highest F1 of all 12 models, with a good balance between catching churners (recall) and not wasting offers (precision); it trains in about two seconds.
- It is not overfitted (train accuracy 0.797 vs test 0.767) and offers feature importances that match the EDA, which makes it easy to explain to business teams.
- The ensembles did **not beat it**: Bagging is essentially tied (F1 0.638) and the boosting models are slightly lower (0.627–0.632). Voting and Stacking have the best ROC-AUC (0.846 and 0.847) and the highest recall (0.877 and 0.930), but at the default 0.5 threshold they flag too many customers as churners (precision 0.457 and 0.424, accuracy 0.691 and 0.647), which gives them the lowest F1 scores (0.601 and 0.583). Stacking also needs about 34 s to train versus about 2 s for Random Forest. A likely reason is that both were fitted with balanced sample weights while their member models already use **class_weight='balanced'**, which corrects for the imbalance twice.
- Differences of about 0.01 between the top models are within noise on a test set with 374 churners, so Random Forest, Bagging and the boosting models should be considered about equal; Random Forest wins on F1, simplicity and speed.
- KNN has the best accuracy (0.788) but misses 42% of churners, which shows why accuracy was not used to choose the model.

Possible future improvements

- Refit Voting and Stacking without the extra sample weights (or tune their decision threshold) to see whether their strong AUC can be turned into a better F1.
- Tune hyperparameters with cross-validated search and compare models with k-fold cross-validation instead of a single split to get more reliable rankings.
- Choose the decision threshold from the real cost of a lost customer versus a retention offer, instead of the default 0.5.
- Try SMOTE or other resampling, and gradient-boosting libraries such as XGBoost / LightGBM / CatBoost.
- Add richer data (usage, support calls, complaints, competitor offers, price changes over time) and explain individual predictions with SHAP values.
- Calibrate probabilities, monitor the model for drift, and run A/B tests of retention offers aimed at the high-risk segments found in the EDA (new month-to-month fiber customers paying by electronic check).

Streamlit application

app.py loads the saved best model (**churn_model.joblib**) and the shared preprocessing module (**churn_features.py**), so a customer entered in the form goes through exactly the same feature creation, encoding, scaling and selection as the training data (verified: the app pipeline reproduces the notebook test-set F1 of 0.6396 exactly). The user enters customer details, and the app shows a clear "Likely to churn / Likely to stay" result, the churn probability with a progress bar, a risk level and possible retention actions taken from the EDA. Run it with **pip install -r requirements.txt** followed by **streamlit run app.py**. Because the model uses balanced class weights, its probability is a risk score that runs higher than the real churn rate.